arXiv:2509.17582v2 [cs.RO] 21 Sep 2026

GeCCo - a Generalist Contact-Conditioned Policy for Loco-Manipulation Skills on Legged Robots

Vassil Atanassov[†,1], Wanming Yu[1], Siddhant Gangapurwala[2], James Wilson[3], Ioannis Havoutis[1]

***Abstract*— Most modern approaches to quadruped locomotion focus on using Deep Reinforcement Learning (DRL) to learn policies from scratch, in an end-to-end manner. Such methods often fail to scale, as every new problem or application requires time-consuming and iterative reward definition and tuning. We present Generalist Contact-Conditioned Policy (GeCCo) — a low-level policy trained with Deep Reinforcement Learning that is capable of tracking arbitrary contact points with a quadruped robot. We shift from task-specific end-to-end learning to a modular hierarchy in which a single planner-agnostic, contact-conditioned tracking policy serves as a robust interface between planning and control. By maintaining stable contact execution under dynamic uncertainty and planner mismatch, the policy enables high-level planners (e.g., handcrafted, learned, or optimization-based) to be composed and swapped without retraining. We demonstrate the scalability and robustness of our method by evaluating on a wide range of locomotion and manipulation tasks in a common framework and under a single generalist policy. These include a variety of gaits, traversing complex terrains (e.g., stairs and slopes) as well as previously unseen stepping-stones and narrow beams, and interacting with objects (e.g., pressing a button, pushing a barrel). Our framework acquires new behaviors more efficiently, simply by combining a task-specific high-level contact planner and the pre-trained generalist policy. Videos: https://vassilatn.github.io/gecco.github.io**

I. Introduction

From dense foliage and forests, to rocky mountains and caves, the morphology of legged robots provides a solution with better agility and utility in all the terrains surrounding us. But realizing this potential requires grappling with the complexity of control: adapting gaits, transitions, and motion strategies to the irregular demands of varied terrains. In the last few years, Deep Reinforcement Learning (DRL) has made legged locomotion controllers significantly more robust, computationally efficient, and capable of ever-more difficult tasks. DRL approaches can now not only traverse natural environments, but actively interact with the world.

While state-of-the-art controllers achieve impressive results, many of them aim to solve just one specific task — for example to jump [1], [2], open doors [3] or manipulate objects [4], [5] to name a few. This typically involves defining the task specifications and reward functions, and fine-tuning their (likely interconnected and co-dependent) scales. As a result, these methods only show a single well-tuned behavior and often do not generalize across tasks. If we want the robot to exhibit new behaviors, or succeed in new unseen environments, we would need to retrain the policy from scratch. This can result in a significant computational burden, especially as many state-of-the-art approaches rely on first training a teacher policy, and then distilling it into a student network [6], [7]. Furthermore, these methods often use a command-space designed for the specific task in mind — for example, this is commonly a linear and angular velocity target [7], goal-position for the robot to reach [8]–[10], desired jumping targets [2], or end-effector targets [5]. Unfortunately, this means that we cannot easily distill all of these controllers into a single policy, unless we predict beforehand and pad for all possible commands, masking out the unnecessary ones for each individual task.

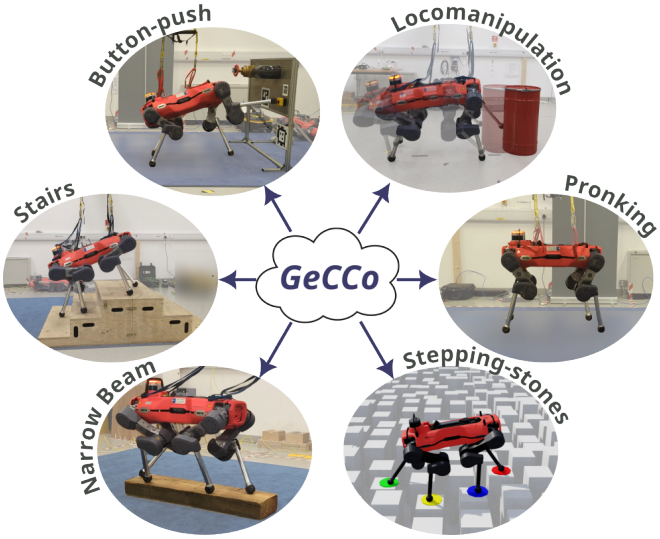

Fig. 1: A selection of diverse and distinct robot behaviors resulting from a single policy trained with GeCCo.

Current state-of-the-art approaches are limited in their generality, as these mostly use the aforementioned command space structure. To overcome this, we propose a generalist contact-conditioned framework, named GeCCo hereafter, which learns to track desired contact locations and durations for each foot. By decoupling the problem into planning and control, we can train a controller that generalizes better than an end-to-end policy, effectively reframing the task as an in-distribution generalization problem. In this work, we focus specifically on the **tracking** problem, and use a simple planner for both training and inference. Despite the planner not taking into account kinematic and dynamic constraints, GeCCo is capable of learning a robust policy that can accurately track the feet locations, even in dynamic, real-

[1]Machine Learning and Articulated Robots Group, Oxford Robotics Institute, University of Oxford, [2]Sony AI, [3]Dyson Institute of Engineering and Technology, [†]Corresponding author: vassilatanassov@robots.ox.ac.uk

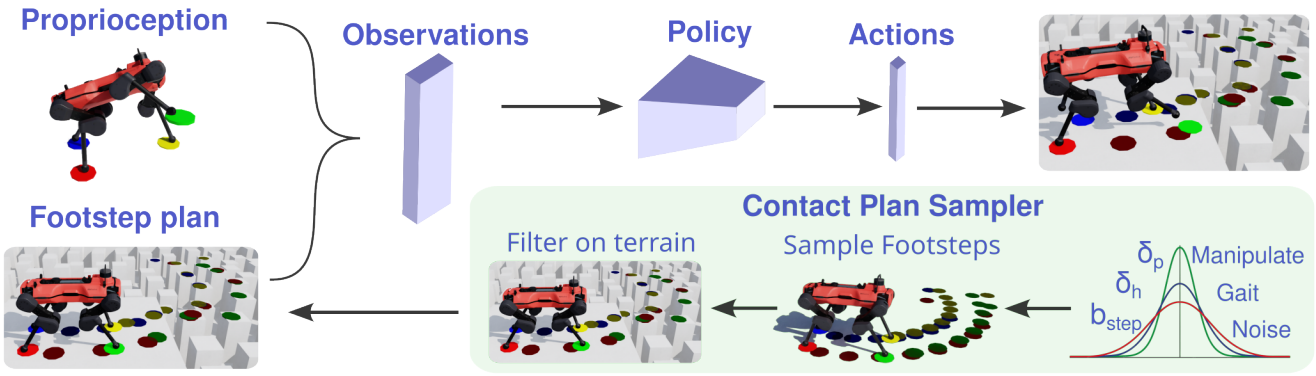

Fig. 2: General scheme of the GeCCo framework. The planner samples a desired contact plan, based on several distributions. The policy then receives the observations, consisting of the proprioception and the current footstep target, and produces desired joint angles as the actions.

world environments. We demonstrate GeCCo's utility on a wide range of tasks on the quadruped ANYmal robot — such as walking, jumping, manipulating objects, and traversing complex terrains (see Fig. 1), and our framework can easily scale to robots with fewer or more contact points. While contact-tracking controllers have been proposed in prior work [11]–[13], to the best of our knowledge, GeCCo is the first one to do this in a general way, across a variety of tasks, locomotive gaits and (non-prehensile) loco-manipulation skills under a single policy. Thanks to our sampling strategy, GeCCo can be trained quickly and efficiently in about half a day on a single GPU.

Our contributions are as follows:

- We present a novel generalist contact-conditioned (GeCCo) policy for learning to track arbitrary foot plans. Using a sampling-based approach to generate desired contact locations, we can train general policies more efficiently.
- We demonstrate that the policy can exhibit different styles and locomotive gaits, and successfully traverse complex terrains (staircases, narrow beams, and stepping stones).
- We combine both locomotion and manipulation capabilities within a single policy, which enables the robot to seamlessly combine walking and interacting with objects using its feet as non-prehensile manipulators.
- We deploy GeCCo in the real world using a simple heuristic-based planner, showing strong robustness and adaptability in tracking the contact plan, despite the planner not considering kino-dynamic constraints.

II. Relevant Work

Legged locomotion: Model-based control methods often decompose the task of locomotion into several subcomponents: perception and localization modules that produce elevation maps and state estimates, high-level planners that generate optimal trajectories for the feet and the center of mass, and low-level controllers (sometimes also hierarchical) that aim to track these trajectories [14], [15].

In the last several years, deep neural networks trained with reinforcement learning have shown incredible performance when it comes to both blind [16], [17] and perceptive locomotion [7], [18]. Most commonly, the locomotion task is framed as a base velocity-tracking or as a goal-conditioned one [8], [10]. Sparse terrains (like stepping-stones), however, have remained particularly difficult for such locomotion controllers. Recent work [19] has shown success by training separate specialist policies for stepping-stones and narrow beams. Other works [20] have done this with a single policy with an attention encoder, but require encountering those environments during training, and have long training times (several days [20]). On the other hand, our proposed GeCCo framework can handle these terrains zero-shot, without training on them.

Reinforcement learning has also been used to learn policies capable of whole-body loco-manipulation using the limbs of the robot. These approaches learn a single task using a dedicated end-to-end policy, to dribble [21] and manipulate objects to a desired location [4]. Recent work [5], [22], [23] has made this more general by learning to track desired positions using the feet of a quadruped robot. However, [5], [22] both learn a specific manipulation policy, and require a separate locomotion policy to walk normally. In contrast, GeCCo can do this naturally under the same framework, allowing a seamless combination of manipulation and locomotion, as shown in Section IV-D.

Contact-tracking: Recently, several works mirrored model-based controllers by learning contact-conditioned policies [11]–[13]. Similarly to MPC locomotion controllers, there is a high-level planner (or heuristically-defined plans) that computes a foothold schedule, which the locomotive policy aims to track. DTC [11] used Trajectory Optimization (TO) to compute a footstep plan for the RL controller to track, which, however, leads to long training times (14 days). WoCoCo [12] learns from manually-defined plans, and shows that the same reward can be used to learn locomotion, manipulation, and other controllers. However, in contrast to our proposed approach, a separate policy is

trained for each task in WoCoCo, and it requires exploration terms to combat the sparsity of the rewards. In [13], a sampling-based planner is learned alongside the tracking policy, allowing both controllers to influence each other as they improve. On the other hand, to shrink the learning space and reduce computational cost, the authors only consider a single bounding gait. In GeCCo, as we sample from a large set of contact locations and timings, the policy can instead achieve a much wider range of gaits.

III. Methodology

A. Contact-commands space

To train the GeCCo contact-tracking framework, at the beginning of each episode we sample $\{\mathscr{R}_i\}_{i=1}^{N_{\text{stages}}}$ for all contact points, where N_{stages} is the number of contact stages.Each $\mathscr{R}_i$ is a target region for a contact point, defined as a sphere in $\mathbb{R}^3$:

$$\mathscr{R}_i = \{\, p_i \in \mathbb{R}^3 \mid \|p_i - c_i\|_2 \le r \,\}, \quad r = 0.1\,\text{m},$$

centered at c_i. For a quadruped, this gives four regions per stage, one for each foot. A contact is satisfied for a foot when its position $p_i \in \mathscr{R}_i$ and the foot is in ground contact. In practice, this collapses to the support surface,

$$\tilde{\mathscr{R}}_i = \{\, p_i \in \mathscr{R}_i \;\mid\; (p_i)_z = z_{\text{ground}} \,\},$$

so contact satisfaction reduces to $p_i \in \tilde{\mathscr{R}}_i$. For loco-manipulation tasks, however, sampling valid contact points directly on an object in $\mathbb{R}^3$ is difficult. To keep the sampling tractable, we drop the ground-contact condition and only require the foot to lie within the tolerance region, i.e. $p_i \in \mathscr{R}_i$. We tested several values for the tolerance region r, and chose 0.1m as it strikes a good balance between accurate tracking and ease of learning.

Each contact point has an associated contact duration, N_{dur}, which defines how long a foot should remain in contact before transitioning to the next stage. To allow for variable stance duration at inference, and to accommodate gaits that require foot-specific timing, we also introduce randomized durations, allowing each foot (or different combinations thereof) to maintain its own counter and progress independently. In typical cases, we use a uniform duration across all four feet, with progression occurring only once every foot has satisfied its assigned contact time.

Sampling: A crucial part of getting a generalist contact-tracking policy is learning to track a wide range of contact points. Using a model-based planner [11], or learning it alongside the policy [13] risks overfitting to that particular planner. Instead, we choose to sample a large set of possible contacts, which is computationally simpler while covering more of the state space. Inevitably, some of these contact points might not be feasible or stable. However, in our experience, this did not impede training, and helped robustify the policy, by exposing it to harder combinations of contacts. In general, we sample N_{stages} deviations of the base position δ_p, and heading deviations δ_h. We then propagate forward the desired base position. The contact points are then sampled from a zero-mean Gaussian distribution $N(0, 0.05\text{m})$ centered around the nominal location of each foot at the corresponding desired base position. We chose this value, as it allows for the robot to see a larger range of points, without sacrificing feasibility by sampling too far from the robot. We refer to Table I for all of the sampling ranges.

To sample a wide range of behaviors, we define several skeleton gaits (trot, pronk, pace, bound, and single-step gait). These provide a structured yet compact basis that captures the principal coordination patterns observed in legged locomotion. From this foundation, we introduce variability by randomizing the contact durations across different feet. This combination allows us to generate a rich space of behaviors that extends far beyond the original skeletons, effectively balancing pre-engineered gait designs with the flexibility to explore diverse and nonstandard gaits. It is worth noting that the policy is not explicitly aware of the gait – it only observes the desired contact points for the current step. At the start of each episode we also sample whether the robot should locomote or manipulate (or a combination of both)[1].

For more complex terrains, like stairs and stepping-stones, we need to adjust the pre-planned footsteps. To make sure the steps lie on a surface, we project them to the nearest horizontal surface, while avoiding being too close to an edge. We also filter out contacts that would be too far away from the body ($\geq$ 1m) to eliminate completely infeasible plans.

TABLE I: Sampling ranges for GeCCo.

Name (Unit)	Sampling Range
Base pos. dev. δ_p (m)	$U(0.0, 0.4)$
Base head. dev. δ_h (rad)	$U(-\pi/6, \pi/6)$
Footstep noise (m)	$N(0.0, 0.05)$
Footstep width b_{step} (m)	$N(0.2, 0.1)$
Contact duration (step)	$U(1, 50)$
Independent feet	Ber(0.1)
Direction	Ber(0.5)
Walk, Manipulate, Both	Binomial$(0.5, 0.4, 0.1)$
Manip. range (x, y, z) (m)	$U(\pm[0.4, 0.3, 0.6])$
Gaits (Trot, Pace, Bound, etc.)	Binomial(0.2)

B. Observation and Action space

We use an Asymmetric Actor-Critic [24] framework, where the critic observations contain privileged information. The actor observations, $o_{\text{actor}} \in \mathbb{R}^{81}$, consist of the base linear and angular velocities, projected gravity vector, foot contact states, foot position (base frame), contact commands error (base frame), a boolean indicating whether the contacts are satisfied (and at the previous step), joint positions, velocities, and previous actions. We also tested a variation that included the elevation around the robot as an exteroceptive observation, but it did not result in better performance (see Section IV-E). The critic ($o_{\text{critic}} \in \mathbb{R}^{110}$) contains the same observations, but without added noise, as well as the foot velocities, the time at which correct contact was made, the elapsed time in the current contact stage, the desired contact duration, a flag to indicate if a given foot should be manipulating, and base height. The actions consist of

[1]We dynamically shift this probability proportionally to the mean contact stage of locomotion relative to manipulation.

the desired joint positions for the 12 joints, relative to the nominal stance configuration.

C. Curriculum

As in prior work [16], [25], we employ a curriculum that automatically increases the difficulty of the environment as the agent learns. Completing at least 10 stages by the end of the episode increments the curriculum level, and reaching fewer than 5 decrements it. The terrain levels are scaled by the curriculum, as in [25], and so are some penalties - as described in Section III-D.

D. Rewards

1) Task Rewards: We use one main event-driven task reward, similar to that of [12] and [26], that consists of four terms. It is worth noting that this is a general reward and does not favor certain gaits, rather it only ensures the correct contacts are reached, as follows:

$$\begin{aligned} r_{\text{task}} = \gamma_{\text{rew}} \Big(\underbrace{n_{\text{corr}}}_{\text{Correct}} + \underbrace{\tfrac{50}{N_{\text{dur}}} \mathbb{1}[n_{\text{corr}} = n_{\text{total}}]}_{\text{Completion bonus}} \Big) \\ -\gamma_{\text{pen}} \Big(\underbrace{n_{\text{wrong}} - n_{\text{corr,prev}}}_{\text{Wrong}} + \underbrace{n_{\text{total}}\, n_{\text{broken}}}_{\text{Correct broken}} \Big) \end{aligned} \tag{1}$$

where n_{corr}, n_{wrong}, n_{total} are the number of currently correct, incorrect, and total contacts (in our case this is four, one for each foot); $n_{\text{corr_prev}}$ are the contacts that were correct at the previous contact stage and have not moved since, n_{broken} are the number of contacts that are no longer correct.

The first two terms reward correct contacts and give a bonus when all contacts are correct, respectively. The latter two terms penalize wrong contacts – this can lead to large negative rewards as soon as the agent receives a new target, so we temporarily discount the penalty as long as the foot remains at the previously correct point [26] – and breaking correct contacts as we want the agent to continuously maintain these contacts. We then use two scales for the different terms: γ_{pen} for the penalties, which is 0 initially, and increases by a factor of $0.25 \times$ curriculum level; and γ_{rew} for the rewards, which is defined as follows:

$$\gamma_{\text{rew}} = \frac{1}{2} \underbrace{\left(1 + \exp(-100\, d_{\text{cntc}}^2)\right)}_{\text{proximity}} \cdot \underbrace{\left(1 + \exp(-(\sigma_{\text{td}}/2)^2)\right)}_{\text{variance}}, \tag{2}$$

where $d_{\text{cntc}} \in [0, 0.1]$m is the distance between the desired and actual contact, and σ_{td} is the variance of the touchdown time for the feet whose contact target has changed. The proximity term encourages stepping closer to the center of the target. The variance term rewards moving feet in synchronicity, and helps with tracking different gaits.

2) Regularization Rewards: To regularize the motion and allow it to transfer to the real hardware, we use several smoothing rewards (see Table II), which have been shown [17], [25] to be important for successful sim-to-real transfer. To discourage dragging the feet when moving from one footstep to the next, we employ a clearance penalty as in [27]. In addition, as the task reward could encourage quick motions, after a certain curriculum level a barrier-function-styled cost [28] kicks in to penalize exceeding foot velocity above $1.75\text{m}/\text{s}$ for locomotion and $0.5\text{m}/\text{s}$ for manipulation.

TABLE II: Reward terms and scales. n_j, n_c, n_f denote the number of joints, collision points, and feet; the smooth barrier function $bf(\cdot)$ is defined in [28].

Reward Name	Function	Scale
Task	Eq. 1	5.0
Base Height	$\exp(-400\|\|h-0.6\|\|^2)$	10.0
Alive Bonus	$\mathbb{1}_{\text{alive}}$	10.0
Flat Orientation	$\|\|g_{\text{proj}}\|\|^2$	-5.0
Joint Acceleration	$\sum_i^{n_j} \ddot{q}_i$	-2.5e-6
Action Rate	$\sum_i^{n_j} \|\|a_i^t - a_i^{t-1}\|\|^2$	-0.3
Energy	$\sum_i^{n_j} \|\|\tau_i \dot{q}_i\|\|$	-5.e-3
Undesired Collisions	$\sum_i^{n_c} \mathbb{1}_{\text{i,contact}}$	-1.0
Foot Clearance	$\sum_{i=1}^{n_f} S(-\sigma.h_i).\|\|v_i^{xy}\|\|$	-5.0
Foot velocity	$\sum_i^{n_f} bf(-1.75 + \|\|v_f\|\|, \mu)$	-30.0

3) Terminations: To ensure the policy is safe to deploy, and to facilitate learning, we terminate upon body contacts, and if the contact stage has not progressed within 8s.

E. Symmetry Augmentation

To learn a more general policy, we sample from many contact plan parameters (see Table I), most of which is done at the beginning of each episode. This can limit the number of unique permutations seen during training, and so to help the learning process, we perform some symmetry augmentation on the data. Following [29], we augment with symmetry around the X-axis, Y-axis and the diagonal XY-axis. In essence, we mirror the observations (and corresponding actions) around those axes, and concatenate them to the original observations. This has both an aesthetic effect and a performance benefit, which we analyze in Section IV-E.

F. Training setup

We use IsaacLab [30] to train the policy for 12k update steps, with 24-step roll-outs across 4096 environments, and 15 epochs and 8 mini-batches per update. We use separate networks for the actor and the critic of size $\text{dim} = [512, 256, 128]$ each, with Mish activations. We use a classifier-based critic [31] with 51 bins to get a more stable value loss. We train on a single RTX 4090 for ~ 15 hours.

IV. Results

A. Experimental setup

We deploy the policy on the ANYmal-D robot. As in sim, the initial footsteps are planned ahead of time, and then adjusted online depending on the terrain. At runtime, the stock proprietary state estimator is used to express desired contact locations in the base frame. When possible, we avoid replanning to isolate and stress-test the tracking policy without corrective feedback. GeCCo was able to track long-horizon plans lasting up to ~ 30s without drift-induced failures. We compare against an end-to-end velocity-tracking baseline, trained as prescribed in IsaacLab [30] for 12k update steps. We try two variants — *Baseline (Partial)* trained on the same terrains as GeCCo (slopes, stairs, rough

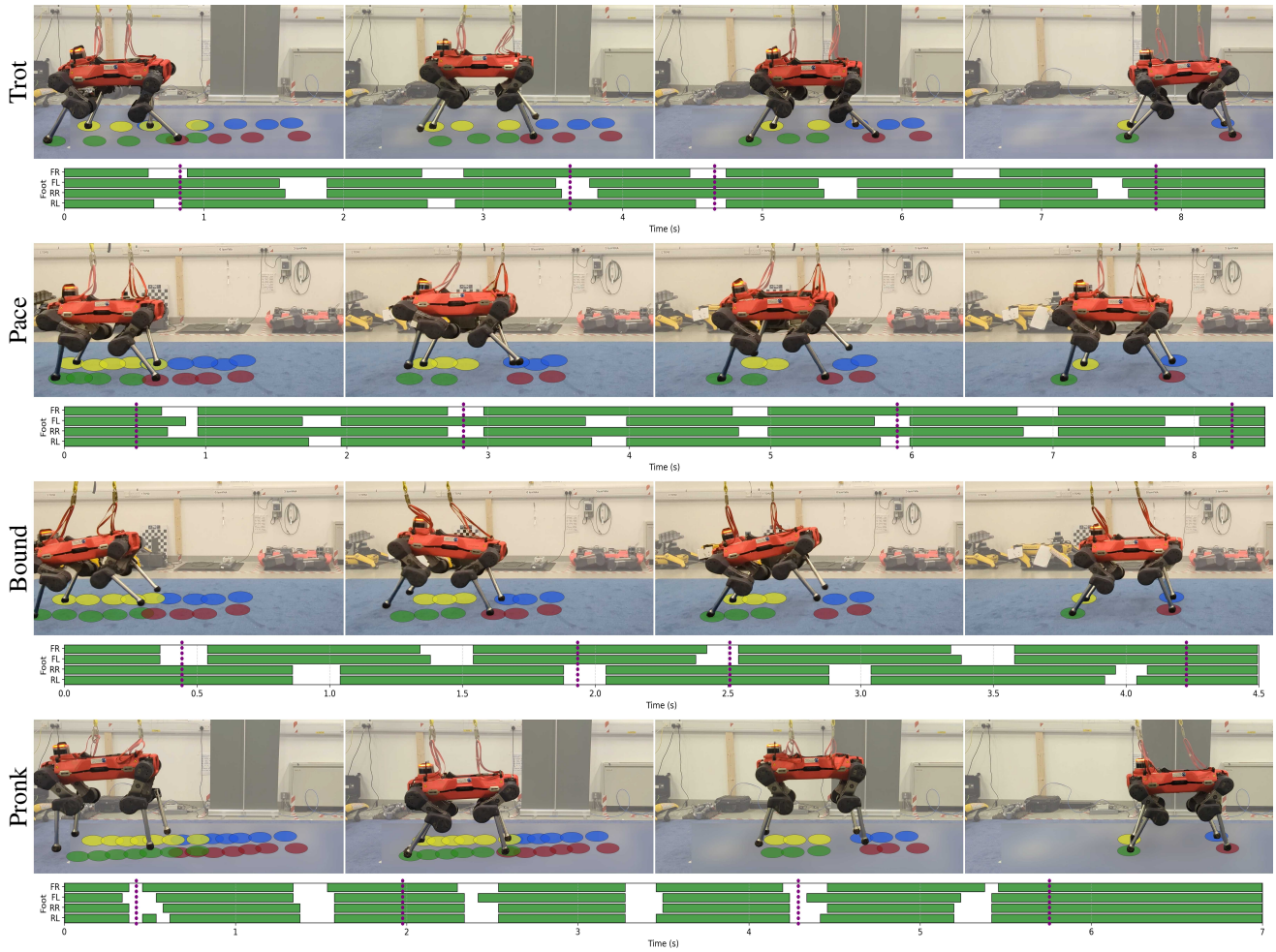

Fig. 3: GeCCo executing multiple different gaits, with their corresponding footfall pattern: *Top row*: Trotting gait with alternating diagonal feet. *Second row*: Pacing gait with alternating left/right side. *Third row*: Bounding gait, with the front feet jumping first, followed by the rear feet. *Bottom row*: Pronking/Jumping gait, with all feet taking off and touching down simultaneously. The circles illustrate the real contact locations.

terrain), and *Baseline (Full)* which includes sparse terrains (stepping stones, beams).

B. Locomotive gaits

In Fig. 3, we show ANYmal perform four different gaits – namely trot, pace, bound, and pronk. Underneath each time-lapse, we show the footfall pattern, using the built-in state estimator. As can be seen, the robot can track different gaits, with little variation between synchronized feet. In the supplementary video, we show that GeCCo can also smoothly transition between the gaits.

C. Complex terrains

To show the versatility of GeCCo, we evaluated its performance on a variety of complex terrains, such as the ones in the IsaacLab suite [30], as shown in Fig. 4. While sparse terrains are notoriously difficult for RL controllers [19], GeCCo can easily traverse these terrains, as well. We tested two variations — stepping stones and a narrow beam, neither of which was seen during training. Nevertheless, as shown in Fig. 4, the policy can successfully traverse them in zero-shot, without any fine-tuning.

In Fig. 5, we report the quantitative results across five different terrains (similarly to those seen in Fig. 4, with *Stairs-Up* and *Stairs-Down* having step width of 0.25m and height of 0.2m) for 3×500 trials. We keep the planner fixed to a default stepping gait of length $\in [0.15, 0.2]$m and width of 0.2m. GeCCo achieves better results than an end-to-end velocity-tracking policy on the staircases, even with a simple footstep planner. On the more difficult, sparse terrains that have not been seen during training (*Stepping-Stones-Easy* (0.8m width stones, 0.05m gap), *Stepping-Stones* (0.35m width stones, 0.2m gap) and *Narrow Beam* (0.30m width)), GeCCo manages to maintain a similar success rate, while the Baseline (Partial) policy fails entirely. Baseline (Full) achieves $\sim 18\%$ success on *Stepping-Stones-Easy*, but still fails to traverse the more difficult ones. Prior work [19] has shown that end-to-end policies require additional curricula, exploration terms, and fine-tuning to successfully learn to

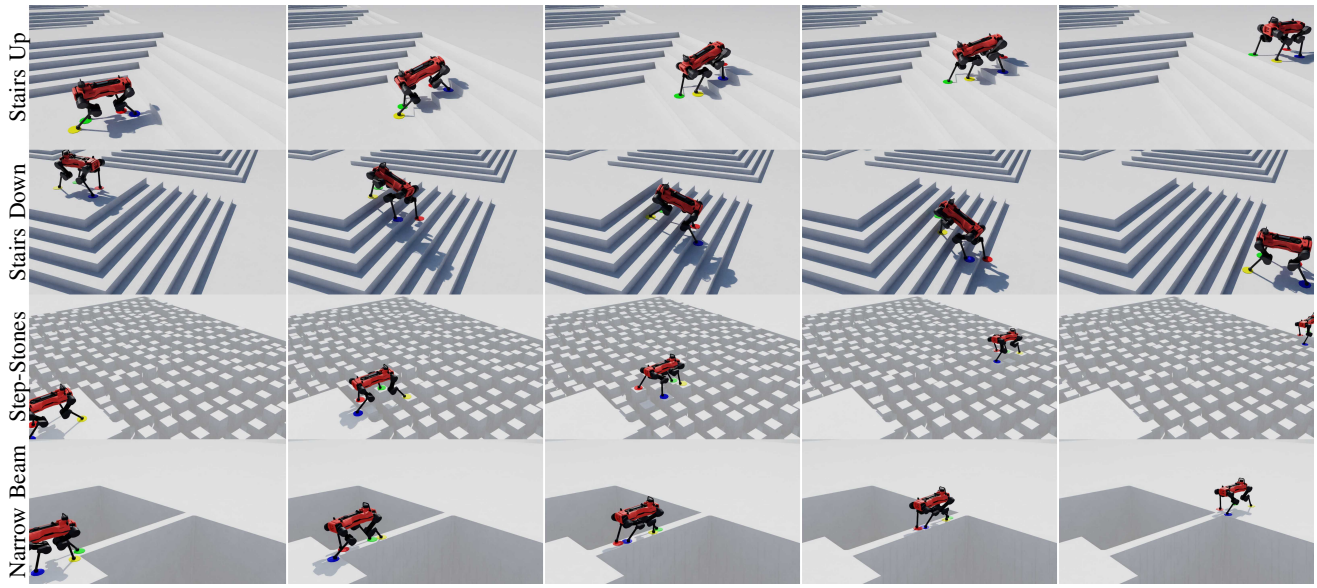

Fig. 4: Traversing various complex terrains using GeCCo: *Top row*: Climbing up stairs; *Second row*: Climbing down a staircase; *Third row*: Traversing unseen stepping stones; *Bottom row*: Walking across an unseen narrow beam.

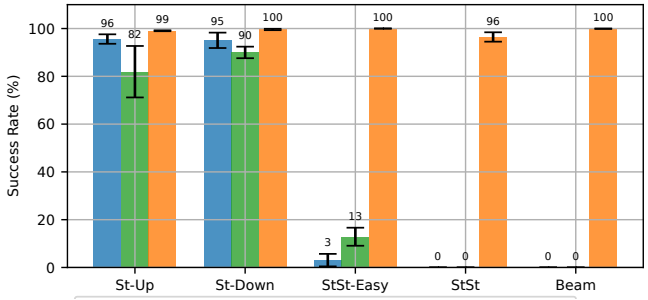

Fig. 5: Results from 3x500 trials in simulation across five difficult terrains, showing the mean success rate and the ±std. Success counts as walking 5m without terminating.

traverse such environments. Due to a lack of open-source implementation and specific details on terrain parameters, we could not directly compare to [11], [19], [20].

In Fig. 6, we demonstrate the successful hardware deployment on these much more difficult terrains. We use a simple footstep planner, which maintains a predefined gait, and uses the existing elevation map to adjust steps to the correct terrain height and away from edges. Despite the planner not considering any kino-dynamic constraints, our results show that GeCCo can track such plans even in adversarial terrains.

D. Loco-manipulation

Our method can also seamlessly combine commands for locomotion and for manipulation together. In Fig. 7, we show four examples of loco-manipulation behaviors, such as reaching for different points in 3D space, walking to a push-button and pressing it using the foot, a non-prehensile task of flipping a box while standing in place, and pushing a barrel continuously while walking. The manipulation target for the push-button task is detected with an AprilTag, while the rest are either teleoperated or predefined. The barrel-push task shows the generalization capabilities of GeCCo — despite the task being outside of the training distribution, the robot can adapt and push the object forward, while also balancing.

E. Ablation

To understand what makes GeCCo work, we run several ablations (Fig. 8), reporting the mean maximum contact stage per episode. In principle the contact plan abstracts the terrain away from the controller, but the robot might still benefit from extra information. **w/ Exteroception** adds height scans as an observation, yet performs slightly worse, likely due to the larger network input – suggesting the contact plan already carries enough information to traverse complex terrains. It also limits generalization: the policy fails on the unseen stepping-stones and narrow-beam terrains, whose height scans are out of distribution. We also evaluated adding the Exteroception in the Critic only, which did not have an effect on the performance (**w/ Exteroception in Critic**). Ablation **w/o Symmetry** removes the symmetry augmentation (Section III-E). This degrades performance two-fold: aesthetically, and more importantly in contact tracking, likely due to differing training samples across feet – confirming that symmetry augmentation aids more general policies.

We then ablate components of the sampling strategy (Table I) to isolate their contribution to robustness (Fig. 9). We evaluate three variants: **Partial Sampling**, which fixes the base position/heading deviation and footstep width during training; **No Sampling**, which further removes footstep noise; and **Single Gait**, which only trains on a trotting gait. Across 10k environments we record the failure rate (falling) and the ratio of wrong footsteps. Without stochastic sampling

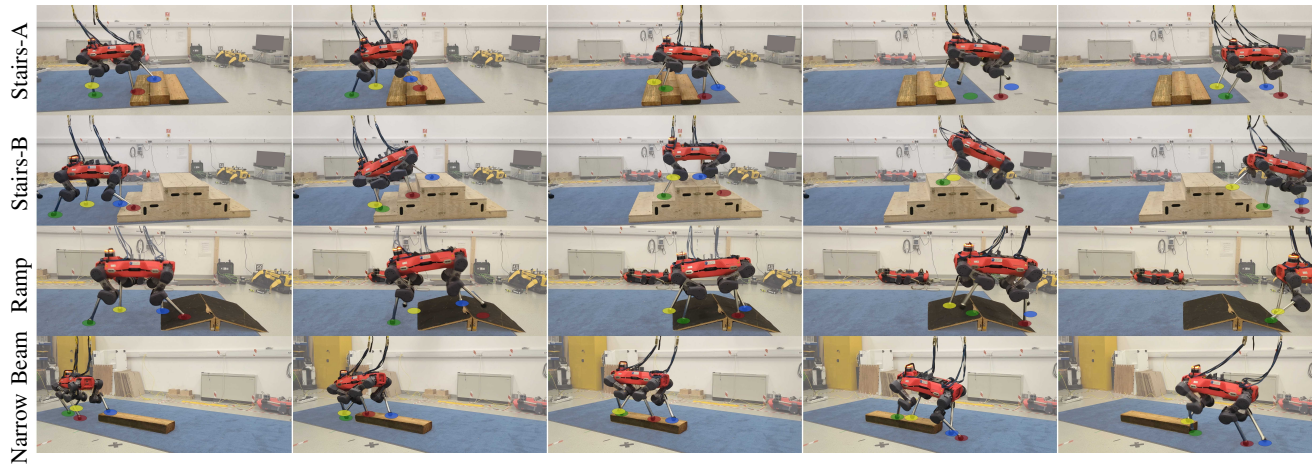

Fig. 6: Traversing various complex terrains using GeCCo: *Top row*: Climbing up small stairs; *Second row*: Climbing up a large staircase; *Third row*: Walking up and down a ramp; *Bottom row*: Walking across a narrow beam.

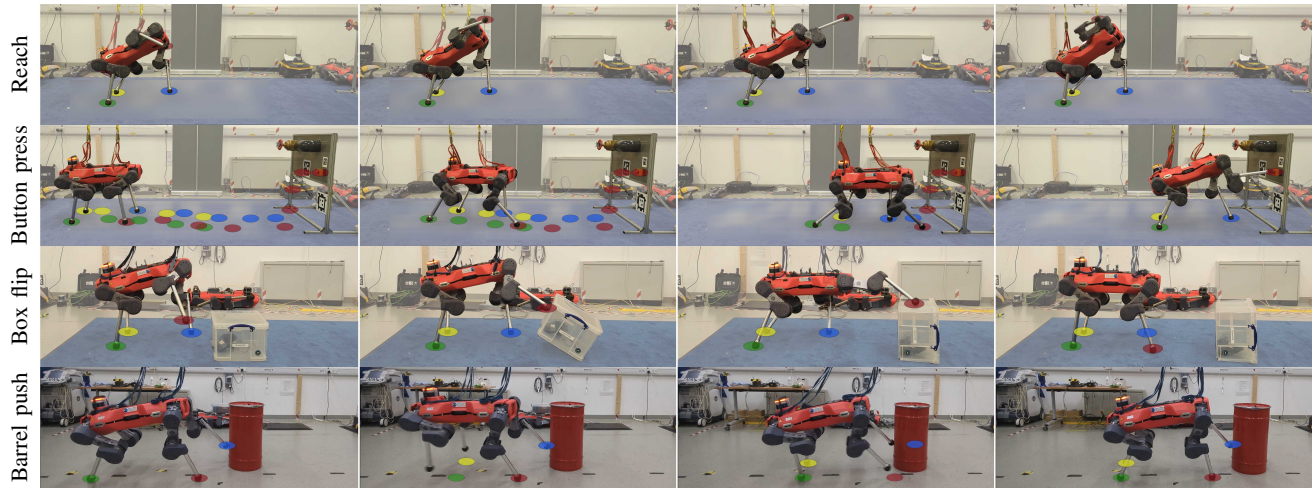

Fig. 7: Loco-manipulation experiments: *Top row*: Reaching out for different locations in 3D space using the front right foot; *Second row*: Button-pressing loco-manipulation task, consisting of walking to the board and then using its foot to press a button; *Third row*: Flipping a box using the robot's foot; *Bottom row*: Pushing a barrel continuously while the robot walks.

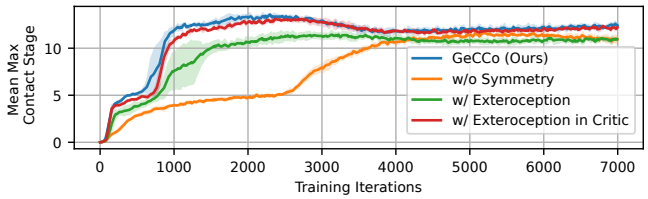

Fig. 8: Ablation study showing the mean maximum contact stage achieved in an episode, with the shaded region showing the variance across 3 random seeds.

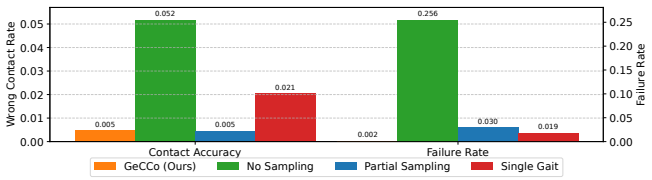

Fig. 9: Ablation bar plots showing the contact tracking accuracy (left) – in terms of median wrong contacts ratio, and the failure rate (right) across 10k envs.

the policy still performs reasonably well, as footstep plans are still adjusted based on the terrain, introducing contact variability. **Partial Sampling** matches GeCCo's tracking accuracy but is less robust, falling $\sim 3\%$ of the time – confirming that stochastic contact perturbations improve robustness. Sampling various gaits is crucial for generalizable tracking: the **Single Gait** baseline shows a higher failure rate and lower accuracy.

V. Conclusion

In this work, we presented GeCCo – a generalist contact-conditioned loco-manipulation policy trained with Deep RL.

We demonstrated how this contact-based decomposition allows us to generalize to previously unseen terrains, like stepping-stones and narrow beams. With the same policy, our method can seamlessly switch between using its feet as non-prehensile manipulators and walking, capable of a variety of different gaits, like pronking, trotting, and pacing. Finally, our sampling-based planner enables doing all of this in a computationally efficient manner, allowing training within a few hours.

There are some limitations to GeCCo that we hope to address in the future. We only consider foot contacts - for more complex terrains the robot might need to use its shins or body to climb. Another limitation is that, while GeCCo can track arbitrary desired contact durations, the swing phase duration is uncontrolled. In future work, we would like to investigate how GeCCo can handle the notion of time better and allow for more precise control over swing phases. Finally, our next step would be to learn high-level planners for various loco-manipulation tasks on top of a pretrained GeCCo policy. As much of the complexity of locomotion has been shifted to this low-level tracker, one could train task-specific planners more easily. This is a direction we are particularly excited about, especially as a way to scale to more complex loco-manipulative interactions.

Acknowledgments

We would like to thank Jacques Cloete, Aristotelis Papatheodorou, Reece O'Mahoney, and other members of the Oxford Robotics Institute for their helpful discussions and help with experiments.

References

[1] N. Rudin, H. Kolvenbach, V. Tsounis, and M. Hutter, "Cat-Like Jumping and Landing of Legged Robots in Low Gravity Using Deep Reinforcement Learning," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 317–328, 2022.

[2] V. Atanassov, J. Ding, J. Kober, I. Havoutis, and C. D. Santina, "Curriculum-Based Reinforcement Learning for Quadrupedal Jumping: A Reference-Free Design," *IEEE Robotics & Automation Magazine*, vol. 32, no. 2, pp. 35–48, 2025.

[3] C. Schwarke, V. Klemm, M. v. d. Boon, M. Bjelonic, and M. Hutter, "Curiosity-Driven Learning of Joint Locomotion and Manipulation Tasks," in *Proceedings of The 7th Conference on Robot Learning*. PMLR, Dec. 2023, pp. 2594–2610.

[4] S. Jeon, M. Jung, S. Choi, B. Kim, and J. Hwangbo, "Learning Whole-Body Manipulation for Quadrupedal Robot," *IEEE Robotics and Automation Letters*, vol. 9, no. 1, pp. 699–706, 2024.

[5] P. Arm, M. Mittal, H. Kolvenbach, and M. Hutter, "Pedipulate: Enabling Manipulation Skills using a Quadruped Robot's Leg," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. Yokohama, Japan: IEEE, May 2024, pp. 5717–5723.

[6] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by Cheating," in *Proceedings of the Conference on Robot Learning*. PMLR, May 2020, pp. 66–75.

[7] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[8] N. Rudin, D. Hoeller, M. Bjelonic, and M. Hutter, "Advanced Skills by Learning Locomotion and Local Navigation End-to-End," 2022, arXiv:2209.12827 [cs].

[9] V. Atanassov, W. Yu, A. L. Mitchell, M. N. Finean, and I. Havoutis, "Constrained Skill Discovery: Quadruped Locomotion with Unsupervised Reinforcement Learning," 2024, arXiv:2410.07877 [cs].

[10] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, "ANYmal Parkour: Learning Agile Navigation for Quadrupedal Robots," 2023, arXiv:2306.14874 [cs].

[11] F. Jenelten, J. He, F. Farshidian, and M. Hutter, "DTC: Deep Tracking Control – A Unifying Approach to Model-Based Planning and Reinforcement-Learning for Versatile and Robust Locomotion," 2023, arXiv:2309.15462 [cs, eess].

[12] C. Zhang, W. Xiao, T. He, and G. Shi, "WoCoCo: Learning Whole-Body Humanoid Control with Sequential Contacts," 2024.

[13] H. Kim, H. Oh, J. Park, *et al.*, "High-speed control and navigation for quadrupedal robots on complex and discrete terrain," *Science RoboticS*, 2025.

[14] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, "Perceptive Locomotion through Nonlinear Model Predictive Control," 2022, arXiv:2208.08373 [cs].

[15] A. Papatheodorou, W. Merkt, A. L. Mitchell, and I. Havoutis, "Momentum-Aware Trajectory Optimisation using Full-Centroidal Dynamics and Implicit Inverse Kinematics," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2024, pp. 11 940–11 947.

[16] J. Hwangbo, J. Lee, A. Dosovitskiy, *et al.*, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[17] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[18] S. Gangapurwala, M. Geisert, R. Orsolino, M. Fallon, and I. Havoutis, "RLOC: Terrain-Aware Legged Locomotion Using Reinforcement Learning and Optimal Control," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 2908–2927, 2022.

[19] C. Zhang, N. Rudin, D. Hoeller, and M. Hutter, "Learning Agile Locomotion on Risky Terrains," 2023, arXiv:2311.10484 [cs].

[20] J. He, C. Zhang, F. Jenelten, R. Grandia, M. BÄcher, and M. Hutter, "Attention-Based Map Encoding for Learning Generalized Legged Locomotion," 2025, arXiv:2506.09588 [cs].

[21] Y. Ji, G. B. Margolis, and P. Agrawal, "DribbleBot: Dynamic Legged Manipulation in the Wild," 2023, arXiv:2304.01159 [cs].

[22] X. Cheng, A. Kumar, and D. Pathak, "Legs as Manipulator: Pushing Quadrupedal Agility Beyond Locomotion," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. London, United Kingdom: IEEE, May 2023, pp. 5106–5112.

[23] J. Cheng, D. Kang, G. Fadini, G. Shi, and S. Coros, "RAMBO: RL-augmented Model-based Whole-body Control for Loco-manipulation," 2025, arXiv:2504.06662 [cs].

[24] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, "Asymmetric Actor Critic for Image-Based Robot Learning," in *Robotics: Science and Systems XIV*. Robotics: Science and Systems Foundation, June 2018.

[25] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning," in *Proceedings of the 5th Conference on Robot Learning*. PMLR, Jan. 2022, pp. 91–100.

[26] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, "Full-Order Sampling-Based MPC for Torque-Level Locomotion Control via Diffusion-Style Annealing," 2024, arXiv:2409.15610 [cs].

[27] G. Kim, D. Kang, J.-H. Kim, S. Hong, and H.-W. Park, "Contact-implicit Model Predictive Control: Controlling diverse quadruped motions without pre-planned contact modes or trajectories," *The International Journal of Robotics Research*, p. 02783649241273645, 2024.

[28] B. Zhang, Y. Zhang, L. Frison, T. Brox, and J. Bödecker, "Constrained Reinforcement Learning with Smoothed Log Barrier Function," 2024, arXiv:2403.14508 [cs].

[29] M. Mittal, N. Rudin, V. Klemm, A. Allshire, and M. Hutter, "Symmetry Considerations for Learning Task Symmetric Robot Policies," 2024, arXiv:2403.04359.

[30] NVIDIA, M. Mittal, P. Roth, *et al.*, "Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning," 2025, arXiv:2511.04831 [cs].

[31] J. Farebrother, J. Orbay, Q. Vuong, *et al.*, "Stop Regressing: Training Value Functions via Classification for Scalable Deep RL," in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024.